Digital Image Processing

Extra Team Project Report

Team 15

50251566 데니스 고레츠키

20206224 이주환

CHUNG-ANG UNIVERSITY

School of Computer Science and Engineering

Contents

Outline of Program

Purpose

Following the requirements for the extra project, the goal of this program is to determine whether the shape in a given image is a Circle or a Triangle. To achieve this, the program implements basic image processing algorithms and utilizes the geometric features of the object to perform the classification.

Functionality Overview

The program classifies the shape in an image through the following sequence of operations:

- **Image Loading**: Loads the input image using the OpenCV library.
- **Grayscale Conversion**: Converts the color image to grayscale to remove color information.
- **Binarization**: Separates the object (white) from the background (black) based on a specified threshold.
- **Contour Extraction**: Finds the coordinates of the object's contour from the binarized image.
- **Feature Extraction**: Calculates the Area and Perimeter from the extracted contour information, and uses these to derive the Circularity.
- **Shape Classification**: Based on the calculated circularity value, the program makes a final classification of whether the shape is a circle or a triangle.

Design of Program

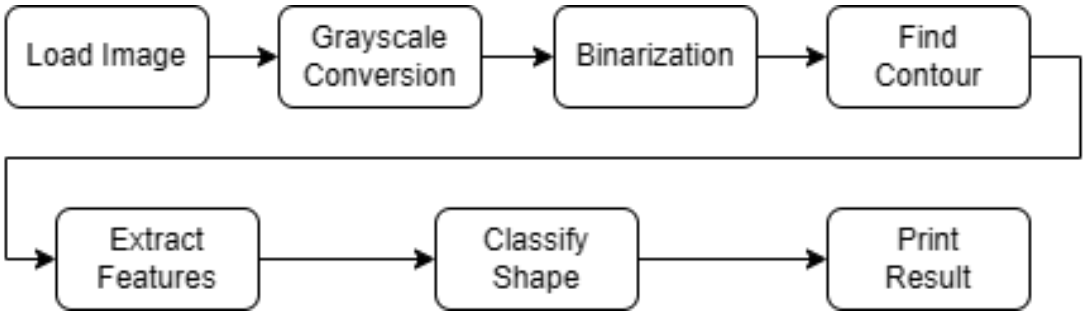
High-Level Design

The program is structured into three main modules based on functionality:

- **Image Processing Module (image.cpp, image.h)**:
    - This module is responsible for the preprocessing pipeline, including image loading, grayscale conversion, binarization, and contour extraction.
    - Adhering to the project constraints, key object detection-related functions (binarization, contour extraction, etc.) were implemented from scratch without using their pre-built library equivalents.

- **Feature Extraction & Classification Module (feature.cpp, feature.h)**:
    - Receives the contour data from the Image Processing Module and calculates key features such as area, perimeter, and circularity.
    - Contains the logic that uses these features to perform the final shape classification.
- **Main Module (main.cpp)**:
    - Orchestrates the overall workflow by calling functions from the other modules in the correct sequence.
    - Prints the final classification result to the console.

Flow Diagram



Implementation Notes

- **Core Classification Logic**: The key to the classification is the Circularity metric. Circularity is calculated using the formula (4 * π * Area) / (Perimeter^2). In theory, a perfect circle has a circularity value of 1.0. In this project, if the value is greater than a certain threshold (0.8), the shape is classified as a circle; otherwise, it is classified as a triangle.
- **Algorithm Details**:
    - **Image Loading (loadImage)**: The implementation uses cv::imdecode to load the image by first reading the entire file into a buffer. This method ensures that images can be loaded reliably, even if the file path contains non-ASCII characters.
    - **Binarization (threshold)**: This function inverts the image, setting the object to white (255) and the background to black (0). This step is crucial preparation for the contour extraction algorithm, which is designed to trace white pixels.

    - **Contour Extraction (findContour)**: A custom contour-tracing algorithm was implemented. It operates by first scanning the image to find a starting pixel of the object. From there, it traces the object's boundary by searching neighboring pixels in a clockwise order, storing the coordinates of the contour until it returns to the starting point.

- **Execution Structure**: The main function in main.cpp is structured with a for loop to sequentially process a total of four image files, from 1.jpg to 4.jpg. For each file, it performs the entire pipeline of image processing, feature extraction, and shape classification before printing the result to the console.

How to execute

To build and run this project, your environment must be configured correctly. The following steps detail the setup process in Visual Studio with the OpenCV library.

1. Configure Project Properties First, configure the project to link with the OpenCV library correctly. Open the project's **Properties** page and ensure the **Platform** is set to **x64**.

- **C/C++ > General > Additional Include Directories**: Add the path to your OpenCV's build\include folder. *(Example: C:\opencv\build\include)*
- **Linker > General > Additional Library Directories**: Add the path to the lib folder that matches your Visual Studio version. *(Example for VS 2019: C:\opencv\build\x64\vc16\lib)*
- **Linker > Input > Additional Dependencies**: Add the names of the OpenCV library files. (Replace XXXX with your version number).
    - opencv_worldXXXX.lib
    - opencv_worldXXXXd.lib

2. Prepare Execution Files

- **Copy DLL Files**: Find the opencv_worldXXXX.dll file in your OpenCV bin folder (e.g., C:\opencv\build\x64\vc16\bin) and copy it into the project's build output folder (e.g., YourProjectFolder\x64\Release).
- **Copy Image Files**: Place the input image files (1.jpg, 2.jpg, etc.) into the main project folder where the .vcxproj file is located.

3. Build and Run

- In the Visual Studio toolbar, set the solution configuration to **Debug** or **Release** and the platform to **x64**.

- Press **Ctrl+F5** or select **Debug > Start Without Debugging** to compile and run the program. The results will be printed to the console window.

Time spent

Planning & Design: ~1 hour

Core Algorithm Implementation (Contour Finding, Feature Calculation): ~3 hours

Visual Studio Environment Setup & Debugging: ~2 hours

Report Writing & Finalization: ~2 hour

Total Estimated Time: ~8 hours

Process of Project and Record of Project Meetings

This extra project was undertaken separately from the main team project. The initial logic was developed in a macOS environment, with subsequent efforts focused on porting the code to a Windows environment to run in Visual Studio.

The most significant challenge was resolving issues related to linking the OpenCV library. Persistent errors related to the inability to load image files arose from incorrect path settings, linker dependencies, and library version mismatches. Through a systematic debugging process, the root cause was identified as an **error in the OpenCV library's linker settings**. After correcting this configuration, the program was confirmed to **operate stably in both Debug and Release modes**, resolving all initial execution issues.

Meeting 1 – Project Kickoff & Initial Development

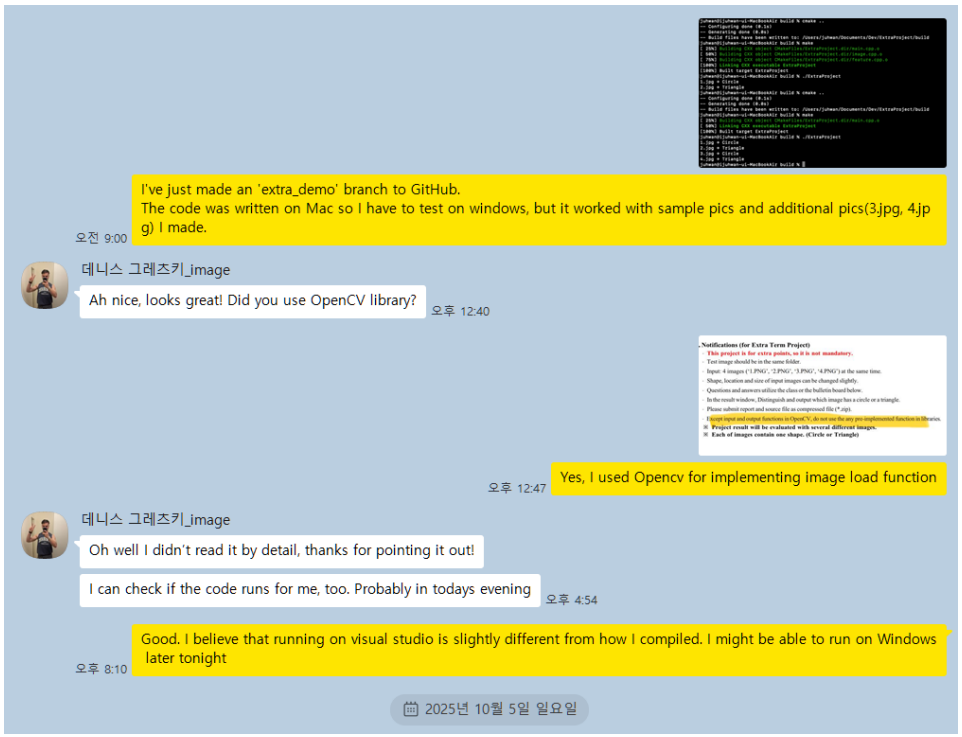
- **Discussion:**
  - Briefly discussed plans for the extra project after class.
  - Decided to use GitHub as the collaboration platform.
  - Assigned roles: One member would implement the initial algorithm while the other would handle testing and debugging.

- **Outcome:**
    - The initial version of the code, which implemented image loading and basic algorithms using OpenCV on macOS, was shared on the extra_demo branch.

Meeting 2 – Windows Testing & Problem Resolution

- **Discussion:**
    - Identified and shared a critical issue where the code failed to load image files in **Debug mode** on Windows, although it worked correctly in **Release mode**.
    - The problem was presumed to be with the cv::imread function. A solution was proposed to change the image loading method to read the file into a buffer and decode it with the cv::imdecode function.
- **Outcome:**
    - The image loading logic was updated using imdecode, and the revised code was pushed to the extra_final branch.
    - The updated code was confirmed to work stably in both Debug and Release modes.

Records



2025년 10월 5일 일요일

데니스 그레즈키_image

Sorry I didn't got to test it yesterday. I will try to do it today.

Would be a nice test to see if I get could get it running, because we need our prof to get it running, too

오후 2:52

There's a lot of time left until the deadline for submission, so don't have to worry.

I've tried the code on Windows yesterday, but I had problems with loading pictures.

I'm at my grandparent's house now, so I can try again on Tuesday afternoon

오후 3:11

데니스 그레즈키_image

Easy, don't worry aswell

오후 3:23

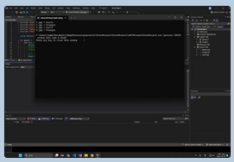
데니스 그레즈키_image

Enjoy the time with your grandparents!

오후 3:24

1

2025년 10월 10일 금요일



1

Hey, how was your long holiday?

By the way, I finally find out the problem with executing Extra Project on Windows. I kept getting errors in "debug mode" and couldn't fix the problem, but it worked fine in "release mode."

오후 10:12

Also, you can find out Visual Studio version in new branch "extra_final"

오후 11:00

2025년 10월 11일 토요일

데니스 그레즈키_image

Holiday was nice, a lot of reating haha. How about yours?

I tried the code from your branch, and I also had problems getting it to run - to be precise: opening the pictures didn't work. I debugged now and I found out, that using the "imdecode" function instead of the "imread" function made it work!

I had to import fstream for that and convert the file path for opencv lib. But, just check it out - I did a commit. If it doesn't work, we can just revert this and look for another solution :)

오후 2:30

Great. I'll try the new code when I get home tonight.

오후 3:12

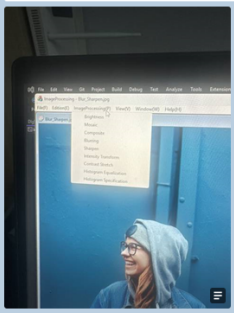
데니스 그레즈키_image

Nice!

오후 4:32

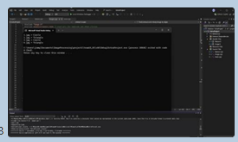
데니스 그레즈키_image

I also found out how to change the template in order to get new menu items, so we could implement the automatic composition and the extra term in there as well



오후 4:33

1



오후 9:38

I pulled code from github and it worked well!

오후 9:39

데니스 그레즈키_image

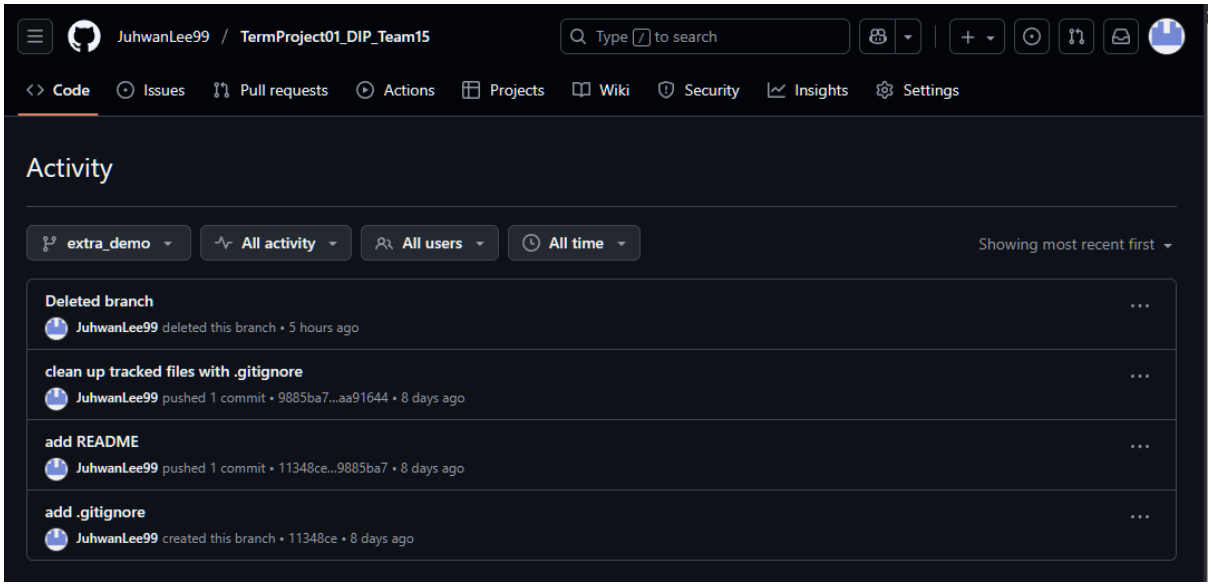
Alright, perfect!

오후 9:39

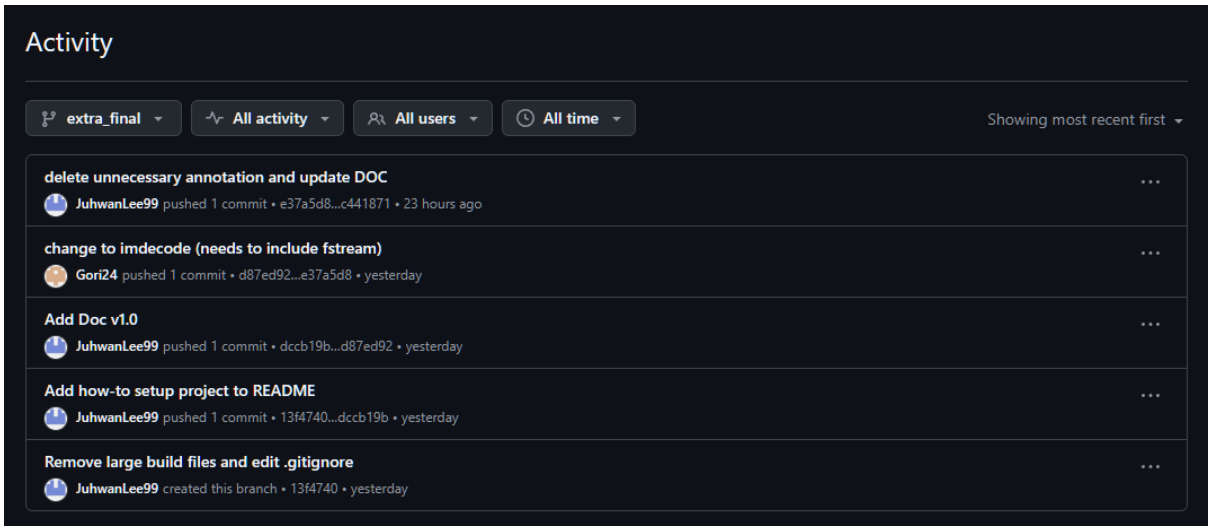
GitHub records

https://github.com/JuhwanLee99/TermProject01_DIP_Team15

- **Initial branch with codes from macOS**



- **Final branch with codes for Visual Studio, Windows**



Member's Thoughts on This Project

Lessons Learned

- **Importance of Algorithms**: Implementing image processing algorithms like contour extraction from scratch, rather than just using a library function, provided a deep understanding of their internal mechanics.
- **Understanding the Development Environment**: This project was an opportunity to experience the differences between operating systems and IDEs. It was a

lesson in navigating the complexities of Visual Studio's project properties. The debugging process reinforced the importance of a systematic approach to problem-solving.

- **Application of Mathematical Principles**: It was insightful to see how a mathematical concept like circularity could be directly applied to classify objects in a real image, connecting theory with practical application.

Future Improvements

- **Support for More Shapes**: The program could be extended to classify more polygons, such as squares and rectangles, by adding additional feature extraction logic.
- **Improved Algorithm Robustness**: The current contour-finding algorithm works well for simple, clean images but may be less effective with noisy or complex images. The algorithm could be enhanced to improve its robustness.
- **Multiple Object Handling**: A future version could include functionality to detect and classify multiple objects within a single image, making the program more practical.